State-Space Feature Encoding and Reward Design for Reinforcement Learning-Based Stock Trading

Jia Yi Ong[1] Yingxin Zhang[2]
[1]Data Science Institute, Columbia University
[2]Department of Computer Science, Columbia University
[1]jo2838@columbia.edu, [2]yz3202@columbia.edu
GitHub: https://github.com/veeannzhang/reinforcement-learning-stock-trading-agent

Dec 15th, 2025

Abstract

Reinforcement learning (RL)–based stock trading remains challenging due to noisy observations, unstable reward signals, transaction costs, and the complex temporal interdependencies between stock prices, the performance of the firms, and macroeconomic conditions. We train Proximal Policy Optimization (PPO) multi-asset stock trading agents that integrate a Mamba-based state-space encoder for efficient long-horizon feature modeling. To induce less-risky trading behavior, we explore a reward signal with turnover regularization that discourages excessive trading while preserving profitability. We compare trading performance against (1) a baseline model with a recurrent policy and value network head and (2) the naive policy of allocating all initial cash equally among stocks and holding them indefinitely. On a testing set with significant regime differences from the training set, the Mamba-based agent outperformed the naive strategy in both annualized and risk-adjusted returns. However, training diagnostics indicate unstable losses during training and the learning of a suboptimal policy. Examining the agents' transaction histories reveals that the agents simply concentrated capital on the stock that had the highest return in the training set, rather than learning a robust trading strategy. To explore interpretability, we conduct feature importance and partial dependence analysis on the sensitivity of the agent's learned policy to market volatility indicators. Overall, our results suggest that while complex sequence models can improve surface-level performance, they alone do not sufficiently address the fundamental challenges of stability and generalization, and often require heavy hyperparameter tuning.

1 Introduction

Reinforcement learning has been widely explored in algorithmic trading, but it is difficult to apply in reality due to noisy and dynamic markets, random reward signals, and execution frictions such as transaction costs and position constraints. Feature representation and reward design are the two most recurring reasons why reinforcement learning trading agents fail. Currently, most of the existing approaches rely on manually designed technical indicators such as moving averages or MACD, and simple feed-forward neural networks that processes each observation independently, with no memory across time. These agents fail to capture longer-term temporal dependencies and changing market behaviors. [Jiang et al., 2017, Liu et al., 2020, Moody and Saffell, 2001]. To solve these problems, recurrent models such as Long Short-Term Memory (LSTM) models are a natural choice, but they often have difficulty converging and are highly sensitive to noise. Additionally, even though LSTMs can store long-term information in theory, they learn to rely more on recent observations in practice. This diminishes the efficacy of the memory mechanism when used on timeseries that exhibit long-term dependencies. At the same time, reward functions based solely on stock prices fail to account for transaction costs, which causes unrealistic and overly frequent trading.

In this project, we address these issues by integrating MAMBA, a selective state-space sequence model, as a feature encoder within the PPO framework to enable stable long-range temporal modeling. We also introduce a turnover-penalized reward that encourages more realistic trading behaviors. Beyond comparing performance between the Recurrent PPO and the MAMBA PPO models, we also conduct a VIX case study on both models to further study the sensitivity of the learned policies to volatility signals.

2 Background

2.1 Related Work

Financial time series exhibit long-range temporal dependencies in the form of persistence in volatility and returns [Mikosch and Stărică, 2004]. To approximate the Markovian assumption underlying most algorithms in RL, many researchers employ deep learning architectures like Long Short-Term Memory (LSTM) networks and Recurrent Neural Networks (RNNs) to learn a sufficient current state representation [Bai et al., 2024]. More recent studies employ advanced transformer architectures that are both temporal- and variable-dependency-aware [Li et al., 2025] or leverage transformer-generated embeddings of temporal dependencies

and complex patterns [Lima et al., 2025] in the stock trading setting. Besides a robust state representation, effective reinforcement learning also hinges on having a well-designed reward that can signal trade-offs between risk and return over different time horizons, which can be tailored toward a specific investment objective [Wang et al., 2025]. It is well known that using arithmetic profit and loss as the reward signal can lead to high-variance strategies, which motivates researchers to design risk-adjusted reward functions. Choudhary et al. [2025] incorporate a differential Sharpe ratio to account for volatility and maximum draw-down to quantify downside risk. Other researchers explicitly penalize transaction costs as a function of value of shares traded [Jiang et al., 2024] or embed transaction costs in the dynamics of the portfolio value [Jiang et al., 2017]. While many works focus on improving reward signaling and function approximation, Bai et al. [2024] find that limited attention has been paid to robustness, specifically the performance of RL agents under different conditions. At the same time, there is a growing focus on Explainable Reinforcement Learning (XRL). Milani et al. [2024] propose a few categories of XRL, which include feature importance via understanding the relationship between actions and input states that is modeled by the policy function.

2.2 Our Contributions

We contribute to the literature by experimenting with applying the MAMBA deep learning architecture [Gu et al., 2023] as a feature extractor for time series stock data, a novel application in the stock trading setting. MAMBA addresses the computational inefficiencies of the Transformer architecture and can scale linearly in input sequence length and achieve fast inference [Gu et al., 2023]. Most importantly, it outperforms Transformers with its state-of-the-art selective state space design. Next, we experiment with a reward design that explicitly penalizes turnover and discourages large portfolio reallocation that do not achieve high profits, which indirectly curbs risky or high-frequency trading without the need to compute risk measures over a past window of data on a moving basis. To test the robustness of our models, we train our trading agents on a pre–COVID-19 period and evaluate their performance on a period that includes the pandemic. This provides insights into the generalizability of RL models across different market regimes. Lastly, we explore the effect of the CBOE Volatility Index (VIX) on the RL agent's trading policy, providing insights into the importance of VIX as a feature.

3 Methodology

3.1 The Trading Environment

We implement a custom stock trading environment to simulate a Markov Decision Process with discrete, daily time steps. Each time step exposes a fixed-length temporal window of current and past market data, including daily prices and stock- or economy-level exogenous variables. It also tracks the current investment portfolio as the RL agent makes trading decisions, i.e., the trader is a price-taker and its actions can only affect the future shares of stock in the portfolio and the cash amount. We place a few restrictions and rules on trading. At each time step, the stock market allows up to 1000 shares of each stock to be bought or sold, at non-integral quantities, and shorting of stocks is prohibited. For each purchase or sale of shares, we apply a transaction cost of 0.5%. Attempts to spend over the current cash limit or sell more shares than in the current portfolio will be clipped to the maximum transaction possible. The environment is initialized with a starting cash value of $5,000. At each time step, the environment exposes data from the current step and 50 past time steps to the RL trader.

3.2 Reward Design

The total asset value at time t is calculated as: $V_t = c_t + \sum_{i=1}^{N} p_{i,t} s_{i,t}$, where c_t is the available cash at time t and $p_{i,t}, s_{i,t}$ are the price and number of shares in the portfolio for stock i at time t. A common formulation of the profit and loss (PnL) reward signal is given by equation (1) below. It is scale-invariant and has low variance, which improves training stability compared to the arithmetic PnL. We use this PnL reward signal to generate a benchmark in our experiments.

$$r_{t+1} = \log\left(\frac{V_{t+1}}{V_t}\right) \tag{1}$$

We also experiment with a reward signal that penalizes turnover, which we measure using the magnitude of portfolio reallocation between two consecutive steps. w_t is the allocation of assets at time t, evaluated at prices at time t. w_{t+1} is the allocation of assets at time $t+1$, evaluated at prices at time t, so that a pure price movement will not result in a change in the calculation of allocation. Note that $\|w_t\|_1, \|w_{t+1}\|_1 = 1$. λ is a regularization parameter that controls the penalty of turnover.

$$r_{t+1} = \log\left(\frac{V_{t+1}}{V_t}\right) - \lambda \|w_{t+1} - w_t\|_2 \tag{2}$$

3.3 Financial Data

The actual trajectories of prices and market variables are based on historical financial data.

3.3.1 Data Engineering

We employ a series of data engineering steps to ensure the RL agent is trained and evaluated on high-quality data. We download stock prices using the yfinance Python package [Aroussi, 2017]. Macroeconomic data are obtained from the Federal Reserve Economic Data (FRED) API, maintained by the Federal Reserve Bank of St. Louis [Federal Reserve Bank of St. Louis, 2025]. We choose the initial set of stocks to cover major industries such as technology, energy, banking, automobile, healthcare, and consumer goods [1]. The initial set of macroeconomic variables is selected to capture various aspects of the economy, such as uncertainty, unemployment,

[1] We included the following stocks for our experiments: AAPL, NVDA, MSFT, AMZN, JPM, XOM, PG, UNH, F, and the SPY index ETF

inflation, interest rates, and general economic activity. Each data series is mapped independently to a full span of dates based on its frequency and date range to reveal any missing values. For each series, missing values are then imputed using the rolling mean of the past 7 days to avoid data leakage. Finally, series of various frequencies are up-sampled to the daily frequency by duplicating values forward to avoid data leakage, then combined into one dataset. Series that are too short are excluded to avoid truncating the dataset. We engineer technical indicators to capture momentum, trend, and volatility for each stock price series. In addition, we apply the sine and cosine function to the day of week to encode cyclical patterns.

3.3.2 Feature Selection

We select the final set of features based on this approach: (1) exclude features with low variation to exclude predictors with low discriminatory information and to reduce the dimension of the feature space and (2) keep only one predictor among many that are highly pairwise correlated to reduce redundancy in the predictive information given to the trading agent. We use Pearson correlation, which, while not suitable for inference on serially correlated data, is a practical feature selection method. The final data set includes 6,730 daily data points for each of the 10 stocks included in our trading environment, ranging from 1999-03-11 to 2025-12-09 (inclusive).

The final technical indicators used for each stock are the 30-day Average True Range (ATR), Moving Average Convergence Divergence (MACD), and the 30-day Relative Strength Index (RSI). The macroeconomic predictors [Federal Reserve Bank of St. Louis, 2025] used are: Consumer Price Index (CPIAUCSL), the Effective Federal Funds Rate (DFF), Initial Jobless Claims (ICSA), the 10-year minus 2-year Treasury Yield Spread (T10Y2Y), the Economic Policy Uncertainty Index for the United States (USEPUINDXD), and the CBOE Volatility Index (VIXCLS) [2].

3.3.3 Train-Test Split

The date '2018-01-01' is used to partition the data set for training and testing. Firstly, it results in a 70.34% (n = 4,734) and 29.66% (n = 1,996) split, which is a common split ratio that balances data between training and evaluation. Secondly, the left partition contains the 2008 recession and the right partition contains the COVID-19 pandemic, each with some surrounding recovery periods to provide temporal context. This serves as a good partition to test the robustness of the RL models.

A quick quantitative analysis is conducted to detect regime change between the two periods. A simple Vector Autoregressive (VAR) model with one lag is used to jointly model the price of each stock and the final set of selected macroeconomic variables. Three separate models are fitted for each stock: on the complete data, on the left partition, and on the right partition. A chi-square likelihood ratio test is then performed, with the null hypothesis that the pooled model is a better fit than having a separate model for each partition. We reject the null hypothesis at the 5% significance level for all stocks. This provides evidence that there is a significant regime change between the split and supports the efficacy of using the split to evaluate model robustness.

[2] Search the series codes on https://fred.stlouisfed.org for more details

3.4 Trading Agent

3.4.1 Proximal Policy Optimization (PPO)

All trading agents are trained using Proximal Policy Optimization (PPO) [Schulman et al., 2017], an on-policy actor–critic algorithm designed to improve training stability in reinforcement learning. The policy gradient estimator is given by

$$\hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t \right] \tag{3}$$

Generalized Advantage Estimation is commonly used for more stable learning: $\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$, where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (the TD residual) and λ quantifies the bias-variance trade-off. The key innovation in PPO is that it optimizes a clipped surrogate objective that constrains policy updates to improve stability. $L^{\text{CLIP}}(\theta) =$

$$\mathbb{E}_t \left[\min\Big(r_t(\theta)\, \hat{A}_t,\ \text{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\, \hat{A}_t \Big) \right] \tag{4}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ measures the change in the probabilities of actions after an update. The value network $V_\phi(s_t)$ is trained using regression with loss function given by:

$$L^{\text{VF}}(\phi) = \mathbb{E}_t \left[\left(V_\phi(s_t) - R_t \right)^2 \right] \tag{5}$$

An entropy loss is included to encourage exploration and prevent premature convergence to a few actions. It penalizes a policy distribution with probability density concentrated in a small region of the action space (low entropy $\mathcal{H}$).

$$L^{\text{ENT}}(\theta) = -\mathbb{E}_t \left[\mathcal{H}\left(\pi_\theta(\cdot \mid s_t) \right) \right] \tag{6}$$

Popular implementations, such as Stable Baselines3 [Raffin et al., 2021], combine the three losses: $L(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\phi) + c_2 L^{\text{ENT}}(\theta)$

3.4.2 Recurrent PPO Baseline

As a baseline model, we use a Multilayer Perceptron (MLP) feature extractor that learns a representation of the raw input state values at the current time step. While the feature extractor is shared, the policy (actor) and value (critic) have distinct network heads. Each head comprises three LSTM layers with 32 hidden states in each layer, followed by two fully-connected layers with 64 units in each layer. This configuration yields a total of 93,269 trainable parameters for the baseline model. For recurrent policies, PPO is applied using truncated rollout sequences, where hidden states are carried across time steps within each rollout but reset between episodes. This enables the policy to leverage temporal memory while maintaining stable and efficient training.

3.4.3 Mamba-Based Encoder

In our novel approach, we employ a Mamba-based feature extractor [Gu et al., 2023] to encode multivariate time-series inputs. All features are first linearly projected onto a shared latent embedding space and then processed by a stack of Mamba blocks. Mamba implements selective state-space dynamics with input-dependent transitions, allowing the model to adaptively propagate or suppress information across time. As a result, feature selection occurs implicitly through the selective state updates, enabling the encoder to emphasize informative signals (e.g., volatility regimes) while filtering noise over long temporal horizons. To improve stability, each Mamba block is preceded by a standard normalization layer and followed by a residual connection. The final hidden state is used as the encoded market representation. Our configuration yields a total of 92,245 trainable parameters for the Mamba model.

3.5 Training and Testing

Normalization statistics (means and variances) are calculated for the features in the training data, and applied to both training and testing data to avoid data leakage. Training occurs synchronously across 10 vectorized, independent environments. In each iteration, the rollout buffer collects 10 (environments) x 64 (steps) state transitions generated from the environment. Each instance of the environment has a randomized starting timestep to increase diversity in experiences. Returns and advantages are calculated on the aggregated rollout buffer. Losses are calculated for backpropagation on the aggregated experience in randomized batches of 32 state transitions over 5 epochs. Training completes after exactly 1 million time steps. We add an early-stopping mechanism with non-binding triggers to safeguard against over-usage of GPU compute, but all training instances reach 1 million steps. We train four variants of reinforcement learning agents from the combination of the two rewards (logarithmic PnL and penalized turnover with $\lambda = 0.005$) and the two model architectures.

4 Experimental Results

4.1 Performance Evaluation

We evaluate the performance of each trading agent using three financial metrics: Compound Annual Growth Rate (CAGR) [3], Volatility [4], and the Sharpe Ratio with zero risk-free rate (a measure of risk-adjusted return) [5]. We also evaluate the naive "buy-and-hold" policy that allocates the initial $5,000 equally among all 10 stocks and holds the shares until the end of the testing period. The results are summarized in Table 1. The Base-PnL, Base-Penalized, and Mamba-Penalized models executed the same strategy: spend all cash on 118.5 AAPL shares at the start and hold them indefinitely. This strategy outperforms the naive policy in both returns and risk-adjusted returns. The fourth agent, Mamba-PnL, traded shares more frequently but on multiple occasions sold shares within a day of buying.

[3] CAGR $= (V_T/V_0)^{252/T} - 1$, where T is the number of trading days in the testing period

[4] Volatility $= \sqrt{252} \cdot \sqrt{\frac{1}{N-1}\sum_{t=1}^{N}(r_t - \bar{r})^2}$, where $\bar{r}$ is the sample mean of daily returns

[5] Sharpe Ratio $= \sqrt{252} \cdot \bar{r}/\sigma$, where σ is the S.D. of daily returns

Table 1: Performance Metrics by Model and Reward

		Baseline Model		Mamba Model	
Metric	Naive	PnL	Penalized	PnL	Penalized
CAGR	0.161	0.277	0.277	0.152	0.277
Volatility	0.201	0.310	0.310	0.300	0.310
Sharpe Ratio	0.843	0.943	0.943	0.622	0.943

Table 2: Metrics for Top 4 Stocks in Training and Testing Periods, in descending order of Sharpe Ratio (Train)

	CAGR		Sharpe Ratio	
Stock	Train	Test	Train	Test
AAPL	0.312	0.276	0.861	0.944
UNH	0.212	0.066	0.747	0.365
NVDA	0.291	0.581	0.718	1.149
AMZN	0.164	0.184	0.546	0.663

4.2 Model Training Diagnostics

While the three models outperform the naive policy, visualizing model training diagnostics reveal signs of poor convergence and unstable learning (Figure 1). We focus our analysis on the diagnostics of the penalized Mamba model. The most notable indicator is the unusual trend of the standard deviation of the Gaussian policy action distribution: it consistently stays near zero for the most part of training before increasing exponentially toward the end, indicating that the policy was mostly deterministic early on and became unstable. Mean episodic returns initially improve but later plateau at a negative level and decline toward the end of training, suggesting convergence to a suboptimal policy or learning stagnation. In contrast, the critic value loss remains low throughout training besides occasional spikes. The explained variance of the value function remains above zero for most of the training and frequently exceeds 0.5, albeit with substantial variability. This indicates that while the value function is fit reasonably well, stable policy improvement is not achieved.

4.3 Analysis

The fact that all three models coincided on the same buy-and-hold trading strategy for AAPL on the test data, combined with insights from the training diagnostics, suggests that they converged to a policy that effectively selects the stock with the highest CAGR and Sharpe Ratio in the training data (Table 2). However, this strategy is only trivial in hindsight: these two metrics alone are not effective predictors of future returns. UNH, for example, has the second-highest Sharpe Ratio

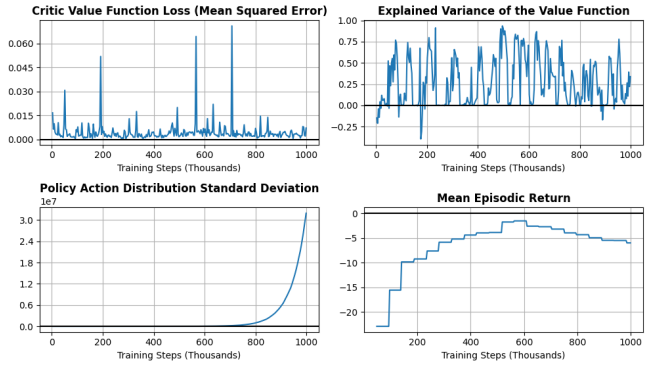

Figure 1: Model Training Diagnostics

during training but experienced a steep decline in returns in the test set. Thus, the models may still have learned non-trivial patterns in the data that enabled them to select a stock that performed well under a regime-shifted holdout dataset. Nevertheless, the training diagnostics indicate clear room for improvement, potentially through improved tuning of hyperparameters such as the turnover penalty, reward scaling, clipping range, and learning rate. Although serial correlation is partially mitigated by batching and shuffling trajectories collected from multiple independent environments, and further reduced by the use of recurrent architectures that model structural temporal dependencies as well as built-in stabilization methods such as advantage normalization in Stable Baselines3, residual correlation may still have contributed to instability during learning. Lastly, the number of parameters in the feature encoders may be too large for our training data, causing the models to be under-fitted.

4.4 Feature Importance: VIX Case Study

1. Hypothesis: We wanted to know whether the agent would adjust its portfolio allocation during high-volatility periods (High VIX) to reduce risk exposure. We test a baseline and a high VIX scenario on the Recurrent PPO model and MAMBA PPO model with the penalized turnover (penalty=0.005) reward function.

2. Methodology: We construct two synthetic scenarios by changing the VIX feature in the trading data while holding all other features constant: replacing VIX with historical mean ($\sim$20.03) in the baseline scenario and with 95th percentile VIX ($\sim$32.92) in the high VIX scenario. We then extract the policy distributions for each stock for both scenarios and compare action means and action standard deviations using bar charts, while plotting distribution curves overlays to examine policy shifts.

3. Results: In both scenarios, the action means and standard deviations are identical. There is minimal distribution shift when VIX changes from average value to 95th percentile value. The barchart in Figure 2 shows identical means for both Recurrent PPO and MAMBA PPO models.

4. Root Cause Analysis: We analyze the VIX feature importance in the first layer of our network, where both models

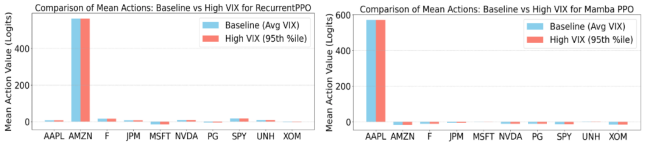

Figure 2: Mean Actions Comparison

have VIX feature importance scores being average across all features. This suggests that the models see the VIX feature, but the signals are likely diminishing going through later layers. We further investigate the maximum absolute difference of policy's output actions, with Recurrent PPO showing 0 and Mamba PPO showing ≈ 0.0002, which is negligible. This confirms that the policies behave the same with average VIX and high VIX. Lastly, we examine the maximum difference between the latent feature vectors, with Recurrent PPO having $\approx$3.97 and Mamba PPO having $\approx$0.006. This shows that the Recurrent PPO feature extractor responds strongly to the high VIX signal, while Mamba feature extractor captures the high VIX signal slightly, but the signals were suppressed for both models at the policy decision level. This is likely because the reward function PenalizedTurnover supressed volatility-driven high turnover behavior in favor of stable policies.

5. Conclusion: Despite increased volatility, neither Recurrent PPO nor Mamba PPO adjusts actions in reaction to high VIX shocks. Although the models can capture volatility information, the models chose not to react because trading frequently is penalized. This suggests that simply using a more powerful model architecture is not enough to encourage the agent to respond to external risk signals.

5 Conclusion

In this project, we studied reinforcement learning-based stock trading with a focus on state representation, reward design, and interpretability. We compared a recurrent PPO baseline model with Mamba-base encoder PPO model with profit-and-loss and turnover-penalized rewards. We found that better feature representations can improve backtest performance, but using a complex model architecture alone does not guarantee that external market signals can be correctly incorporated into trading decisions. Using complex architectures also comes with the burden of requiring large datasets and comprehensive hyperparameter tuning to ensure convergence to the optimal policy. Beyond performance, we analyzed feature importance and partial dependency to show that while the models learned the volatility signals at the feature level, the signals are essentially muted at policy outputs, showing a disconnect between feature learning and decision-making. Aside from model hyperparameters, future work could further experiment with various starting conditions, such as initial cash and shares, trading costs, and leveraged stock trading. To fully evaluate the Mamba architecture, rewards that penalize risk and draw-down should also be tested.

References

Ran Aroussi. yfinance: Yahoo! finance market data downloader. https://github.com/ranaroussi/yfinance, 2017. Accessed: 2025-12-06 to 2025-12-10.

Yahui Bai, Yuhe Gao, Runzhe Wan, Sheng Zhang, and Rui Song. A review of reinforcement learning in financial applications. *arXiv preprint arXiv:2411.12746*, 2024. URL https://arxiv.org/html/2411.12746v1#S4.F2.sf3.

Harsh Choudhary, Anil Orra, Keshav Sahoo, et al. Risk-adjusted deep reinforcement learning for portfolio optimization: A multi-reward approach. *International Journal of Computational Intelligence Systems*, 18(1): 126, 2025. doi: 10.1007/s44196-025-00875-8. URL https://link.springer.com/article/10.1007/s44196-025-00875-8.

Federal Reserve Bank of St. Louis. Federal reserve economic data (fred). https://fred.stlouisfed.org/, 2025. Accessed: 2025-12-06 to 2025-12-10.

Albert Gu, Tri Dao, Étienne Michaud, Atri Kamath, Aidan Hooper, and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. URL https://arxiv.org/pdf/2312.00752.

Yifu Jiang, Jose Olmo, and Majed Atwi. Deep reinforcement learning for portfolio selection. *Global Finance Journal*, 62: 101016, 2024. ISSN 1044-0283. doi: 10.1016/j.gfj.2024.101016. URL https://doi.org/10.1016/j.gfj.2024.101016.

Zhengyao Jiang, Dixing Xu, and Jinjun Liang. A deep reinforcement learning framework for the financial portfolio management problem. *arXiv preprint arXiv:1706.10059*, 2017.

Yifan Li, Xu Dong, Zhuang Wu, Jing Gao, Tianqi Zhang, and Lina Yu. Reinforcement learning with temporal and variable dependency-aware transformer for stock trading optimization. *Neural Networks*, 192:107905, 2025. doi: 10.1016/j.neunet.2025.107905. URL https://doi.org/10.1016/j.neunet.2025.107905.

Adriano Marabuco de A. Lima, Adriano Lorena Oliveira, and Cleber Zanchettin. Chronosrl: Embeddings-based reinforcement learning agent for financial trading. *Procedia Computer Science*, 264:211–220, 2025. doi: 10.1016/j.procs.2025.07.132. URL https://doi.org/10.1016/j.procs.2025.07.132.

Xiao-Yang Liu, Zhe Wang, et al. Deeptrader: A deep reinforcement learning approach for trading. *arXiv preprint arXiv:2004.06627*, 2020.

Thomas Mikosch and Cătălin Stărică. Nonstationarities in financial time series, the long-range dependence, and the igarch effects. *The Review of Economics and Statistics*, 86 (1):378–390, 2004. URL https://www.jstor.org/stable/3211680.

Stephanie Milani, Nicholay Topin, Manuela Veloso, and Fei Fang. Explainable reinforcement learning: A survey and comparative review. *ACM Computing Surveys*, 56(7):168, 2024. doi: 10.1145/3616864. URL https://doi.org/10.1145/3616864.

John Moody and Matthew Saffell. Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 2001.

OpenAI. Chatgpt 5.2. https://chat.openai.com/, 2025. Large language model.

Antonin Raffin, Ashley Hill, Adam Ernestus, Adam Gleave, Anssi Kanervisto, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations, 2021. URL https://github.com/DLR-RM/stable-baselines3.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL https://arxiv.org/pdf/1707.06347.

Feng Wang, Shicheng Li, Shanshui Niu, Haoran Yang, Xiaodong Li, and Xiaotie Deng. A survey on recent advances in reinforcement learning for intelligent investment decision-making optimization. *Expert Systems with Applications*, 282:127540, 2025. doi: 10.1016/j.eswa.2025.127540. URL https://doi.org/10.1016/j.eswa.2025.127540.

Team Member Contributions

Both authors contributed equally to the literature and technical review, building and training RL agents and feature extractor architectures, and writing the manuscript. Jia Yi (jo2838) implemented the trading environment and reward functions, performed the data engineering, analysis, and preparation of financial data, and evaluated model training and performance. Yingxin (yz3202) conducted the VIX case study, analyzed and interpreted the feature importance of the VIX index.

Statement on the Use of GenAI

OpenAI's ChatGPT 5.2 [OpenAI, 2025], assisted in certain aspects of the project's development. It was used as an advanced search engine and learning aid during the exploratory phase to conduct an initial literature review and learn about the technical tools required. Any original work or ideas incorporated into our work have been appropriately cited. While we developed most of the code ourselves, ChatGPT was used to generate a small number of utility functions. In-code citations are provided for functions generated by ChatGPT. All AI-generated code has been thoroughly tested and reviewed.